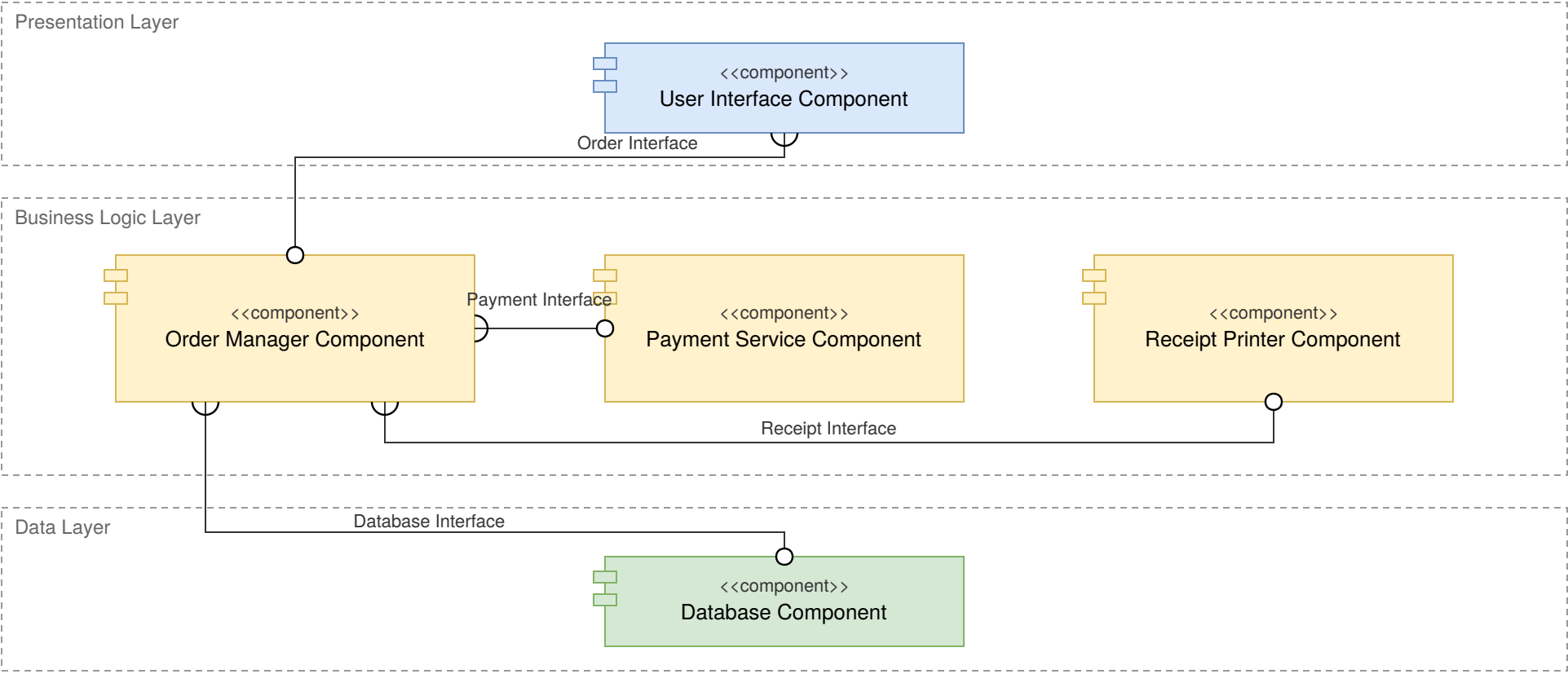


Lab 3 - UML Component Diagram: Self-Service Coffee Kiosk System

Architecture: Layered | R Hemanth Reddy (PES1UG24AM390)



- Provided Interface (Ball)
-) Required Interface (Socket)

Lab 3: Component Modelling & Architectural Pattern Selection

Scenario: Self-Service Coffee Kiosk System | Student: R Hemanth Reddy (PES1UG24AM390)

Architecture Selection: We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

The system is organized into three layers: a Presentation Layer (Touchscreen UI), a Business Logic Layer (Order Manager, Payment Service), and a Data & Hardware Interface Layer (Menu & Pricing Data Store, Receipt Printer Interface). Each layer only communicates with the layer directly below it, keeping the design predictable and easy to reason about.

Two Reasons for This Choice

1. Single self-contained device: A kiosk runs as one deployed unit at a fixed physical location, not a distributed system with independently scaling parts. Microservices would introduce network latency and operational overhead (service discovery, inter-service messaging) with no real benefit here, since there is nothing to scale independently on a single terminal.
2. Clear separation between UI, order logic, and hardware: Coffee type/size selection can change (e.g. seasonal menu items) without touching payment or printing logic, and the printer or card reader hardware can be swapped without affecting the ordering UI, since each concern sits in its own layer behind a defined interface.

Security Advantage

Card payment logic is isolated entirely within the Payment Service component in the Business Logic Layer. The Touchscreen UI never has direct access to card data or the card reader hardware; all payment requests must pass through the well-defined `IPaymentProcessing` interface. This limits the attack surface, since a compromised UI layer alone cannot directly reach payment card data or the payment hardware.

Performance Benefit

Because all layers run within a single local application on the kiosk (not over a network, as microservices would require), calls between the Order Manager, Menu Data Store, and Receipt Printer Interface are in-process or local hardware calls rather than network round-trips. This keeps order placement and receipt printing responsive, which matters directly for customer wait time at a busy café counter.